

CareerConnect

30-Day / 120-Hour Full-Stack Development & Learning Roadmap

Goal: Build a portfolio-ready CareerConnect application while understanding every feature through **What** → **Why** → **How** → **Code** → **Test** → **Interview Explanation**.

Time	Focus
45 min	Concept and architecture — what we are building, why it is needed and how the data flows.
2 hr	Implementation — build the feature step by step.
45 min	Testing and debugging — verify success, failure and edge cases.
30 min	Revision, interview learning, Git commit and next-day notes.

WEEK 1 — Backend Foundation & Authentication

Day 1 — Project Foundation

Daily Task	Understand CareerConnect architecture; review Candidate, Recruiter and Admin roles; organize backend/frontend structure; configure Express, TypeScript, dotenv and CORS; create a health API.
Kya Seekhna Hai	API basics, frontend vs backend, request/response lifecycle, environment variables, project architecture.
Expected Output	Backend runs successfully and GET / returns a health response.
Interview Learning	What is Express? What is a REST API? What is middleware? Why use TypeScript?

Day 2 — Backend Architecture

Daily Task	Separate app/server; create routes, controllers, services, middleware and utils; add consistent API responses, 404 handling and centralized error handling.
Kya Seekhna Hai	Separation of concerns, middleware chain, controller vs service, scalable folder architecture.
Expected Output	Clean backend foundation ready for feature modules.
Interview Learning	Why use controllers and services separately? How does centralized error handling work?

Day 3 — Prisma + PostgreSQL Setup

Daily Task	Configure Prisma and PostgreSQL; understand schema, generated client and migrations; verify database connection.
Kya Seekhna Hai	ORM, relational databases, migrations, Prisma Client.
Expected Output	CareerConnect backend connected to PostgreSQL.
Interview Learning	What is an ORM? What is a migration? SQL vs NoSQL?

Day 4 — Database Design

Daily Task	Design User, Profile, Company, Job and Application models; define IDs, foreign keys, enums and relationships.
Kya Seekhna Hai	One-to-one and one-to-many relationships, primary/foreign keys, normalization.
Expected Output	Initial database schema ready for migration.
Interview Learning	Explain relationships and why the schema is designed this way.

Day 5 — User Registration

Daily Task	Create registration API with Zod validation; check duplicate email; hash passwords with bcrypt; save user through Prisma.
------------	---

Kya Seekhna Hai	Validation, password hashing, HTTP status codes, business rules.
Expected Output	POST /api/auth/register works successfully.
Interview Learning	Why hash passwords? What is bcrypt? Why validate on the backend?

Day 6 — Login + JWT

Daily Task	Create login API; verify email/password; compare bcrypt hash; generate JWT with expiration; return safe user data.
Kya Seekhna Hai	Authentication, JWT structure and token lifecycle.
Expected Output	POST /api/auth/login returns a valid token.
Interview Learning	Authentication vs authorization? How does JWT work?

Day 7 — Auth Middleware + RBAC

Daily Task	Create auth middleware and role middleware for Candidate, Recruiter and Admin; protect selected routes.
Kya Seekhna Hai	Authorization, RBAC, middleware order, 401 vs 403.
Expected Output	Protected APIs enforce role permissions.
Interview Learning	Explain RBAC and the difference between 401 and 403.

WEEK 2 — Core Backend Features

Day 8 — Candidate Profile API

Daily Task	Create read/update profile endpoints for skills, experience, education, location and other candidate data.
Kya Seekhna Hai	Relational updates, request validation, safe profile ownership.
Expected Output	Candidate profile CRUD is available.
Interview Learning	PUT vs PATCH? How do you update related data safely?

Day 9 — Company Module

Daily Task	Allow recruiters to create, read and update their companies; enforce ownership before changes.
Kya Seekhna Hai	Resource ownership and authorization rules.
Expected Output	Company CRUD works for authorized recruiters.
Interview Learning	How do you prevent one user from editing another user's data?

Day 10 — Job CRUD API

Daily Task	Create APIs to create, list, read, update and delete jobs; connect jobs to companies/recruiters.
Kya Seekhna Hai	REST conventions, CRUD design, relational queries.
Expected Output	Complete Job CRUD backend.
Interview Learning	Explain POST/GET/PUT/DELETE design decisions.

Day 11 — Job Search + Filters

Daily Task	Add keyword, location, job type and experience filters plus pagination.
Kya Seekhna Hai	Query parameters, pagination, database query design and indexes.
Expected Output	Professional searchable jobs endpoint.
Interview Learning	Offset vs cursor pagination? Why are indexes useful?

Day 12 — Job Application Flow

Daily Task	Allow candidates to apply to jobs; prevent duplicate applications; save initial application status.
Kya Seekhna Hai	Business rules, unique constraints and transactions.
Expected Output	Candidate can apply to a job once.
Interview Learning	How do you prevent duplicate records? When would you use a transaction?

Day 13 — Application Management

Daily Task	Allow recruiters to view applications for their own jobs and update statuses: Applied, Shortlisted, Interview, Rejected, Hired.
Kya Seekhna Hai	State workflows, resource ownership and authorization.
Expected Output	Recruiter application workflow is complete.
Interview Learning	How would you model and validate a status transition?

Day 14 — Backend Testing + Cleanup

Daily Task	Test every API using Postman/Thunder Client; cover invalid input, missing resources, unauthorized access and duplicates; clean responses and errors.
Kya Seekhna Hai	API testing, debugging, edge cases and error design.
Expected Output	Stable backend MVP.
Interview Learning	Describe your debugging process and how you test an API.

WEEK 3 — Next.js Frontend

Day 15 — Next.js Frontend Foundation

Daily Task	Set up App Router structure; create shared layout, navigation, reusable components, API client, types and environment configuration.
Kya Seekhna Hai	Next.js App Router, Server vs Client Components, component architecture.
Expected Output	Frontend shell is running and connected to backend configuration.
Interview Learning	Next.js vs React? Server Component vs Client Component?

Day 16 — Login + Registration UI

Daily Task	Build forms with validation, loading and error states; integrate registration and login APIs.
Kya Seekhna Hai	Form state, validation and API mutations.
Expected Output	Users can register and log in from the UI.
Interview Learning	Controlled vs uncontrolled inputs? How do you handle API errors?

Day 17 — Protected Frontend Routes

Daily Task	Persist auth state and protect dashboard/profile/application routes; implement logout.
Kya Seekhna Hai	Auth persistence, cookies vs localStorage and route protection.
Expected Output	Unauthorized users cannot access protected UI.
Interview Learning	Where should tokens be stored and what are the security trade-offs?

Day 18 — Candidate Dashboard

Daily Task	Build dashboard summary for profile completion, application counts and important status information.
Kya Seekhna Hai	Dashboard composition, data fetching, loading/error states.
Expected Output	Candidate dashboard MVP.
Interview Learning	How do you structure a dashboard with multiple API requests?

Day 19 — Profile + Resume UI

Daily Task	Build profile editor and resume upload flow with file type/size validation.
Kya Seekhna Hai	multipart/form-data, file validation and form composition.
Expected Output	Candidate can manage profile and resume.
Interview Learning	How does multipart/form-data work? How do you validate uploads?

Day 20 — Jobs Listing UI

Daily Task	Create responsive jobs page with reusable job cards, pagination, loading skeletons and empty states.
Kya Seekhna Hai	List rendering, component reuse and UX states.
Expected Output	Professional responsive jobs listing.
Interview Learning	Why are loading, error and empty states important?

WEEK 4 — Complete User Workflows

Day 21 — Job Details + Apply

Daily Task	Build dynamic job details page and connect apply action with confirmation and error handling.
Kya Seekhna Hai	Dynamic routes, route parameters and mutations.
Expected Output	Complete candidate job-view-to-apply flow.
Interview Learning	How do dynamic routes work in Next.js?

Day 22 — Search + Filters UI

Daily Task	Connect search, location and other filters to backend query parameters; synchronize pagination; add debouncing where useful.
Kya Seekhna Hai	URL search params, debouncing and query state.
Expected Output	Functional job search experience.
Interview Learning	What is debouncing and why use it for search?

Day 23 — My Applications

Daily Task	Build application history page with job/company information, dates, statuses and filters.
Kya Seekhna Hai	API-driven UI, data transformation and status rendering.
Expected Output	Candidate can track every application.
Interview Learning	How do you map backend status values to reusable UI components?

Day 24 — Recruiter Dashboard

Daily Task	Show posted jobs, application counts and workflow summaries with quick actions.
Kya Seekhna Hai	Role-based UI and data aggregation.
Expected Output	Recruiter dashboard MVP.
Interview Learning	How do you build different dashboards for different roles?

Day 25 — Recruiter Job Management

Daily Task	Build create/edit job forms, validation, active/closed state and delete confirmation.
Kya Seekhna Hai	Complex forms, CRUD UI and mutation handling.
Expected Output	Recruiter can fully manage jobs.
Interview Learning	Optimistic vs pessimistic updates?

Day 26 — Recruiter Applications UI

Daily Task	Create application management screen with candidate details, resume access and status updates.
Kya Seekhna Hai	Permissions, UI state synchronization and workflow design.
Expected Output	Recruiter can process applications end-to-end.
Interview Learning	How do you keep UI consistent after a mutation?

WEEK 5 — Admin, Production & Deployment

Day 27 — Admin Dashboard

Daily Task	Build admin controls for users, companies, jobs and moderation actions.
Kya Seekhna Hai	Full-stack RBAC and secure administration.
Expected Output	Admin dashboard MVP.
Interview Learning	How do you secure admin-only APIs and UI?

Day 28 — Notifications + Email

Daily Task	Create notification flow for application events and integrate email for important updates.
Kya Seekhna Hai	Event-driven thinking, asynchronous operations and notification reliability.
Expected Output	Application status notifications work.
Interview Learning	How would you scale notifications using queues/background jobs?

Day 29 — Production Hardening

Daily Task	Review validation, CORS, secrets, authorization, rate limiting strategy, error boundaries, database performance and responsive polish.
Kya Seekhna Hai	Web security basics, performance and production readiness.
Expected Output	Production release candidate.
Interview Learning	How did you secure and optimize the application?

Day 30 — Final Testing + Deployment

Daily Task	Run end-to-end tests; fix remaining bugs; prepare README; configure production environment variables; deploy frontend, backend and database; verify live flows.
Kya Seekhna Hai	Deployment, environment management, logs, monitoring basics and architecture explanation.
Expected Output	Live, portfolio-ready CareerConnect application.
Interview Learning	Give a complete architecture walkthrough from UI request to database response.

Final 30-Day Checklist

- A working full-stack application using Next.js, TypeScript, Express, Prisma and PostgreSQL.
- Candidate, Recruiter and Admin roles protected through authentication and RBAC.
- Jobs, companies, profiles and applications implemented as complete workflows.
- Search, filters, pagination, resume upload and notification concepts implemented.
- Validation, error handling, security review and production-readiness practices applied.
- Frontend and backend deployed with environment variables correctly configured.
- README and project architecture ready to explain in interviews.

Important working rule: Do not blindly copy code. For every feature, first understand what it does, why it is needed, how data flows through the system, then implement and test it.